

GitHub:

<https://github.com/janaashraf888/rowadmasrtasks.git>

Part 1:

1. Why does defining a custom constructor suppress the default constructor in C#?

When you define a custom constructor, C# does not automatically generate a parameterless (default) constructor. This is because the compiler assumes you are handling initialization explicitly and prevents unintended use of an uninitialized object.

2. How does method overloading improve code readability and reusability?

Method overloading allows multiple methods with the same name but different parameters. It improves readability by logically grouping similar operations and enhances reusability by allowing the same method name to handle different types or numbers of arguments.

3. What is the purpose of constructor chaining in inheritance?

Constructor chaining allows a derived class to call a base class constructor. It ensures proper initialization of the base class before the derived class adds its own initialization, maintaining object consistency.

4. How does new differ from override in method overriding?

- `override` modifies a base class method marked as virtual or abstract, replacing its behavior in the derived class.
- `new` hides a base class method without affecting polymorphic behavior; the base version is still accessible via a base reference.

5. Why is ToString() often overridden in custom classes?

Overriding `ToString()` provides a meaningful string representation of an object, which is useful for debugging, logging, or displaying object information.

6. Why can't you create an instance of an interface directly?

Interfaces define a contract without implementation. Since they contain no actual code to create an object, you cannot instantiate them directly; you must implement them in a class or struct first.

7. What are the benefits of default implementations in interfaces introduced in C# 8.0?

Default implementations allow adding new methods to interfaces without breaking existing

implementations. They improve code maintainability and flexibility by providing shared functionality directly in the interface.

8. Why is it useful to use an interface reference to access implementing class methods?

Using an interface reference promotes loose coupling, supports polymorphism, and allows switching implementations without changing the code that uses the interface.

9. How does C# overcome the limitation of single inheritance with interfaces?

C# allows a class to implement multiple interfaces, enabling the class to inherit behavior from multiple sources while still supporting single inheritance for classes.

10. What is the difference between a virtual method and an abstract method in C#?

- **Virtual method:** Has a default implementation in the base class but can be overridden in derived classes.
- **Abstract method:** Has no implementation in the base class and must be implemented in derived classes.

Part 2:

1. What is the difference between class and struct in C#?

- **Class:** Reference type, allocated on the heap, supports inheritance, can have destructors.
- **Struct:** Value type, allocated on the stack, does not support inheritance (except from interfaces), lighter and faster for small data.

2. If inheritance is a relation between classes, clarify other relations between classes.

- **Association:** One class uses or contains another (e.g., a Car has an Engine).
- **Aggregation:** A “whole-part” relationship where the part can exist independently (e.g., a Team has Players).
- **Composition:** A strong “whole-part” relationship where the part cannot exist without the whole (e.g., a House has Rooms).
- **Dependency:** A temporary relationship where one class depends on another to perform a task (e.g., method parameter usage).

Part 3 Bonus:

1. What is chaining to the Base?

Chaining to the base class means calling a constructor of the base class from a derived class constructor. This ensures that the base class is properly initialized before the derived class adds its own initialization. In C#, this is done using the `: base(parameters)` syntax.

2. Why don't structs support inheritance? (Memory allocation)

Structs are value types and stored on the stack, which provides fast allocation and deallocation. Inheritance requires a reference type hierarchy and dynamic memory behavior, which conflicts with the fixed-size, stack-based memory model of structs. Structs can implement interfaces but cannot inherit from other structs or classes.

3. Does overloading or creating multiple versions of a function save memory? (Performance – Method Table)

Method overloading itself does not directly save memory. Each overloaded method exists in the method table (vtable) separately. However, it improves code readability and reusability without affecting the memory of the objects. Performance can be slightly optimized at compile time through early binding, but overloading does not reduce memory usage.

4. What is a Modular Monolithic design?

A modular monolithic design organizes an application into separate modules within a single codebase. Each module has a specific responsibility, but all modules are deployed together as a single executable or assembly. It combines the simplicity of monolithic deployment with some modular structure for maintainability.

5. Early Binding vs Late Binding (Static Binding vs Dynamic Binding)

- **Early binding / Static binding:** Method calls are resolved at compile time. This improves performance because the compiler knows exactly which method will run. Examples: calling a non-virtual method or overloaded method.
- **Late binding / Dynamic binding:** Method calls are resolved at runtime. This allows polymorphism and flexibility, such as calling an overridden virtual method or using reflection, but it is slightly slower due to runtime resolution.

Static Binding (Early Binding):

Static binding occurs when the method call is resolved at **compile time**. The compiler knows exactly which method will be executed, so the call is direct and fast.

- Typically happens with **non-virtual methods, private methods, or overloaded methods**.
- Advantage: Faster execution due to compile-time resolution.

- Limitation: Less flexible; cannot take advantage of polymorphism.

Dynamic Binding (Late Binding):

Dynamic binding occurs when the method call is resolved at **runtime**. The program decides which method to execute based on the **actual object type**, not the reference type.

- Typically happens with **virtual or overridden methods**.
- Advantage: Supports polymorphism and flexible, extensible design.
- Limitation: Slightly slower execution due to runtime lookup.